

UVA CS 4774: Machine Learning

Part 2: Quiz + QA

Related:

- Part 1 [[HERE](#)]
- Course Announcement
Page [[HERE](#)]

Dr. Yanjun Qi

University of Virginia
Department Of Computer Science

09/30/2025 Assignments

- HW2 is due this coming Sunday midnight!
 - Using HW1 code pieces as components;
 - If you struggle with HW1, please contact TA @Haochen ASAP
- HW1 grading is work-in-progress,
 - Grades will be released by next Tuesday class time
 - We posted the guide from TA in Canvas
- Course vote:
 - **New Survey that needs your vote on**
 - 1. back to lecture in-person twice a week?
 - 2. If not, best way to use the in-person session:
 - Quiz to continue
 - + Project discussions
 - Interested in Shark Tank alike setup? idea screening, pitch talk, demo ...

Project Process

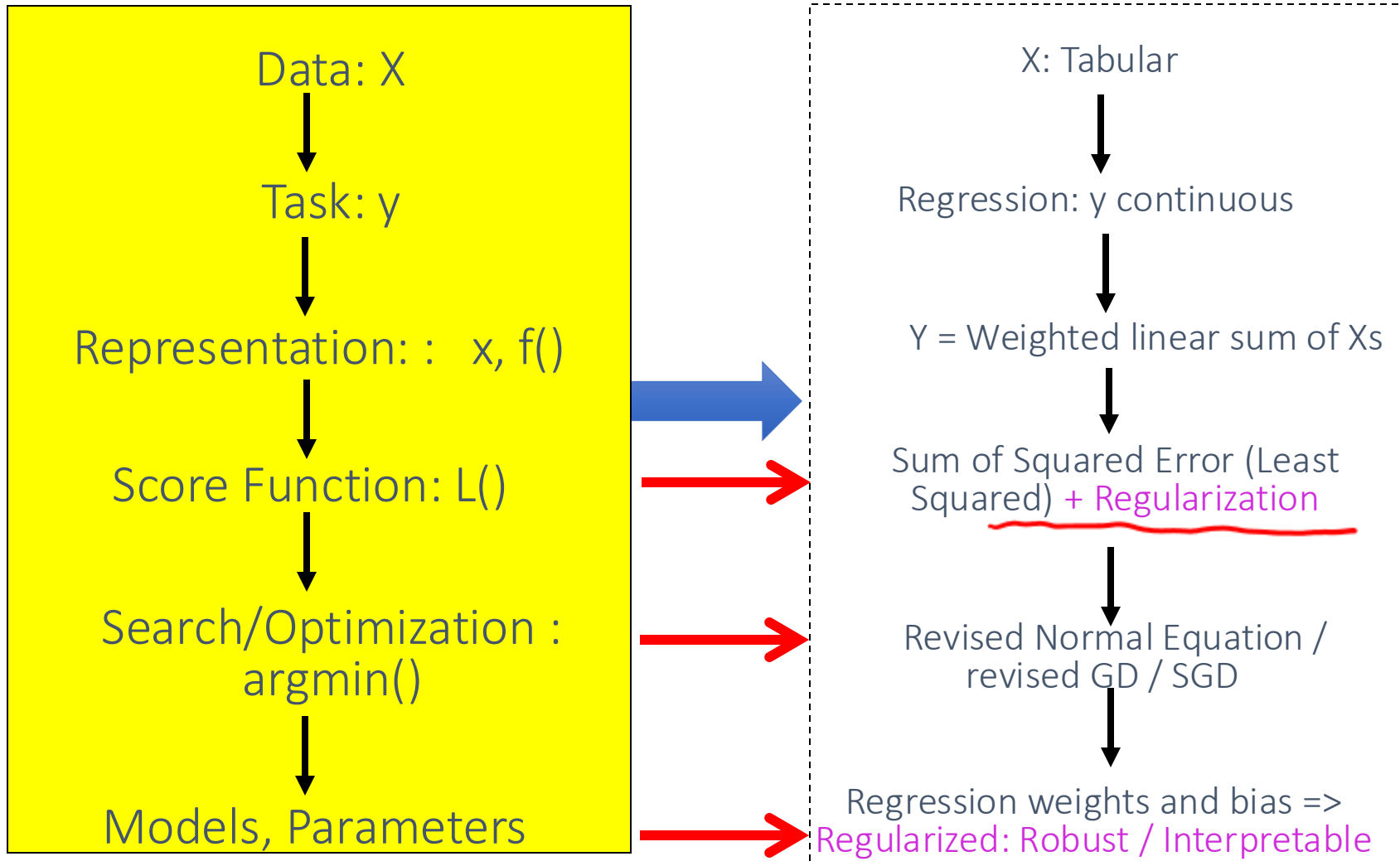


- Format:
 - Team, individual ...
 - Shark Tank alike Screening ? – https://en.wikipedia.org/wiki/Shark_Tank
 - Next week – Idea collection
- Final deliverables:
 - (1) Code (Github PR to course project repo)
 - (2) Poster presentation class wide (Date: TBD)
 - (3) Video Demo (TBD)

09/30/2025 Roadmap

- TA to go over HW1
- One UVA ML club to introduce their setups and projects
- Q5
- Review Q4
- Review QA for L5-L7

L7: Regularized multivariate linear regression

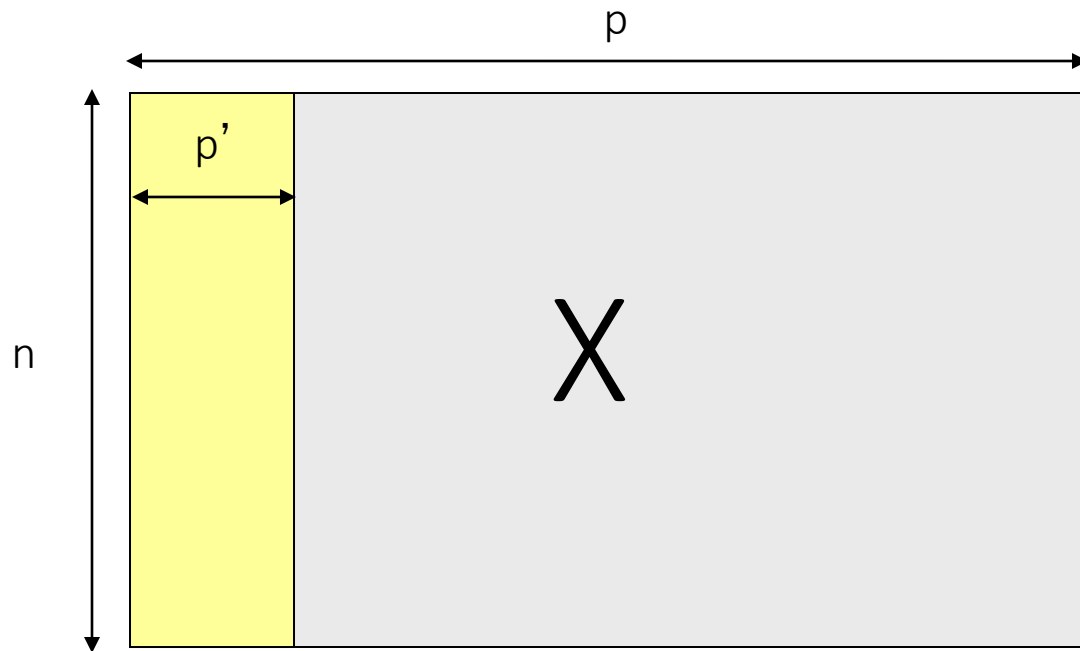


L7: Regularized multivariate linear regression

We aim to make our trained model

- 1. Generalize Well
- 2. Computational Scalable and Efficient
- 3. Trustworthy: Robust / Interpretable
 - Especially for some domains, this is about trust!

Large p , small n : How? $p \rightarrow p' \Rightarrow$ easy to understand



Regularized multivariate linear regression

• Model: $\hat{Y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \cdots + \hat{\beta}_p x_p$

• LR estimation: $\arg \min_{\beta} \sum \left(Y - \hat{Y} \right)^2$

• LASSO estimation: $\arg \min \sum_{i=1}^n \left(Y - \hat{Y} \right)^2 + \lambda \sum_{j=1}^p |\beta_j|$

• Ridge regression estimation: $\arg \min_{\beta} \sum_{i=1}^n \left(Y - \hat{Y} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2$

Error on data

+

Regularization

8/54

Ridge Regression / L2 Regularized Regression

$$\beta^* = (X^T X)^{-1} X^T \bar{y}$$

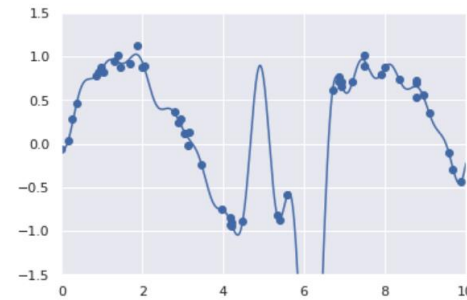


- If not **invertible**, a classical solution is to add a small positive element to diagonal

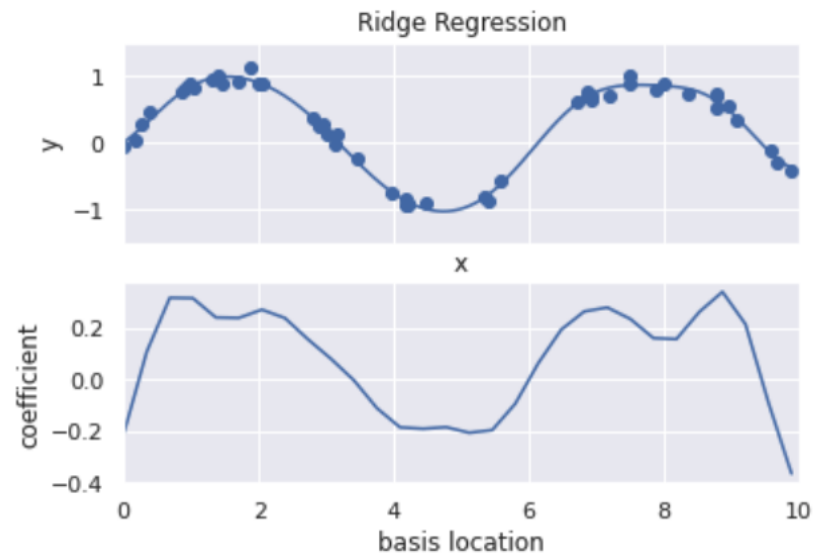
$$\beta^* = (X^T X + \lambda I)^{-1} X^T \bar{y}$$

Overfitting: Can be Handled by Regularization

A **regularizer** is an additional criteria to the loss function to make sure that we don't overfit. It's called a **regularizer** since it tries to keep the parameters more normal/regular



```
from sklearn.linear_model import Ridge
model = make_pipeline(GaussianFeatures(30), Ridge(alpha=0.1))
basis_plot(model, title='Ridge Regression')
```



WHY and How to Select λ ?

- 1. Generalization ability
 ➔ k-folds CV to decide
- 2. Control the bias and Variance of the model (details in future lectures)

L2: Squared weights penalizes large values more

L1: Sum of weights will penalize small values more

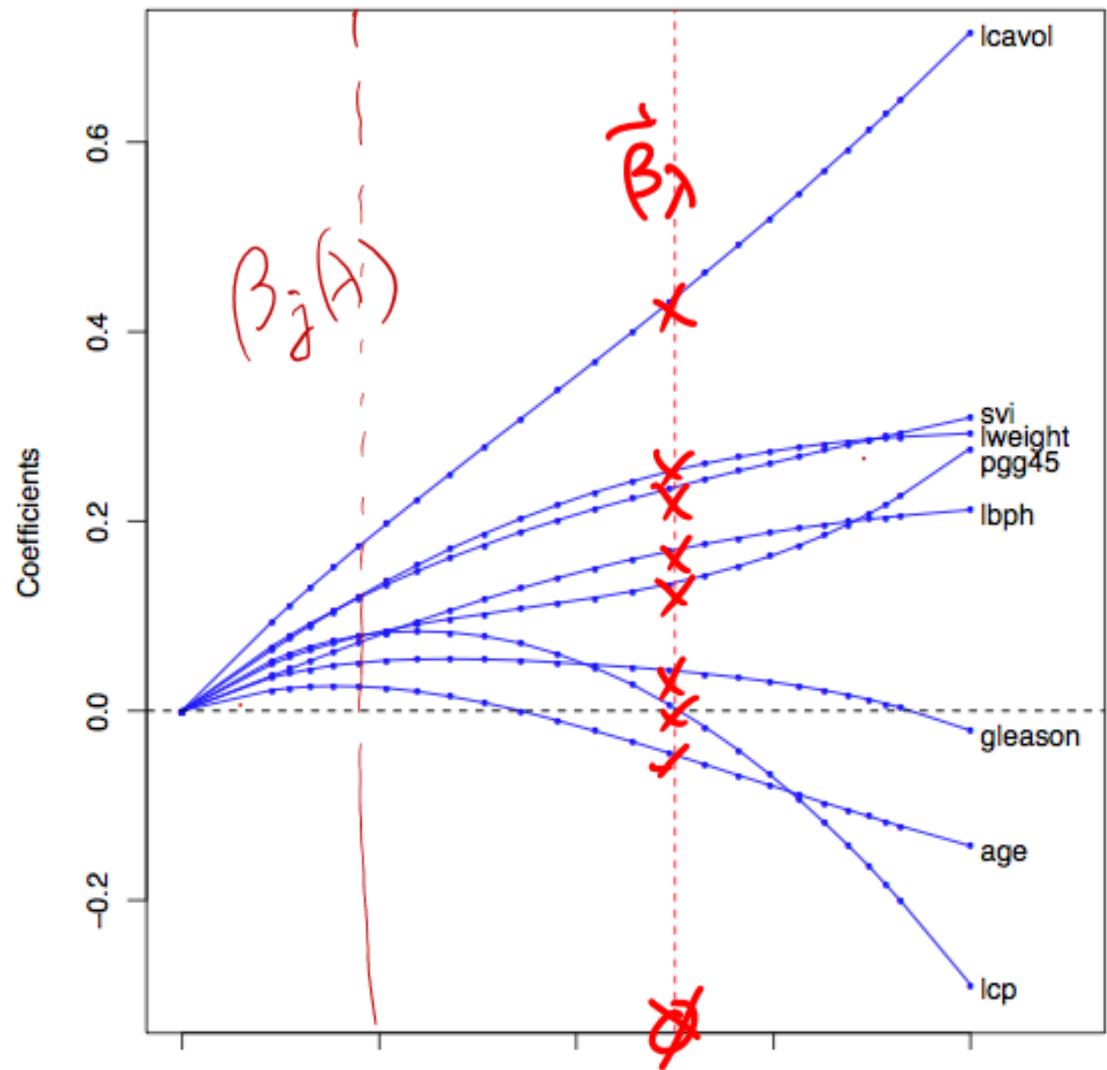
$$\sum_j \beta_j^2$$

$$\sum_j |\beta_j|$$

Regularization path of a Ridge Regression

When $\bar{X}^T \bar{X} = I \Rightarrow \frac{1}{1+\lambda} \beta_{OLS}$

Weight Decay



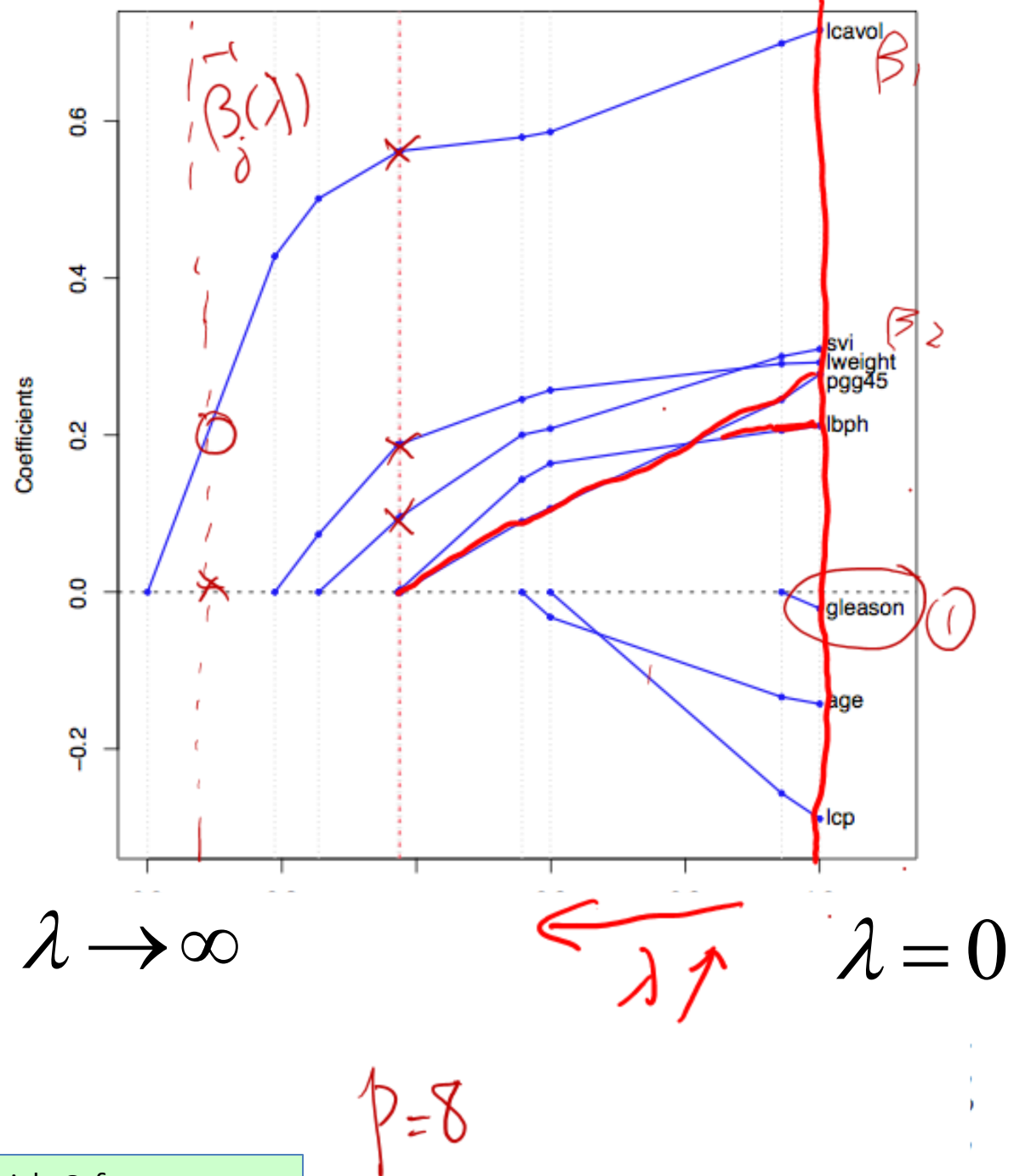
WHY and How to Select λ ?

$$\lambda \rightarrow \infty$$

$$\lambda = 0$$

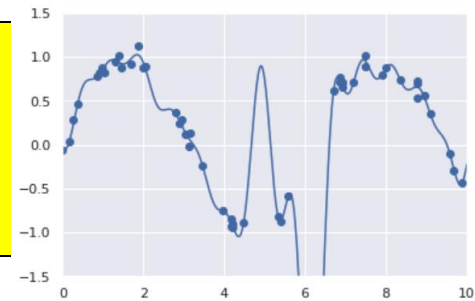
Regularization path of a Lasso Regression

when varying λ ,
how β_j varies.



Overfitting: Can be Handled by Regularization

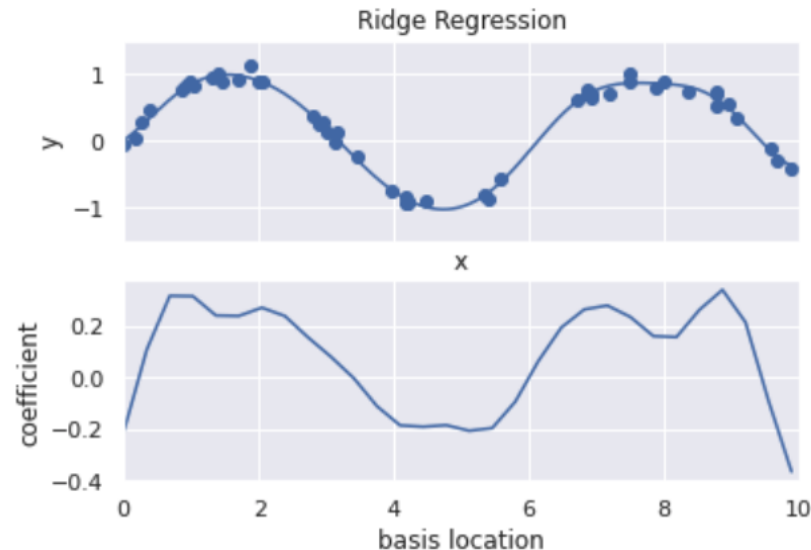
A **regularizer** is an additional criteria to the loss function to make sure that we don't overfit. It's called a **regularizer** since it tries to keep the parameters more normal/regular



code-run:

https://github.com/qiyanjun/2025Fall-UVA-CS-MachineLearningDeep/blob/main/notebook/L7_regularizedRegression_06_LinearRegression.ipynb

```
from sklearn.linear_model import Ridge
model = make_pipeline(GaussianFeatures(30), Ridge(alpha=0.1))
basis_plot(model, title='Ridge Regression')
```



(Extra) Lasso (least absolute shrinkage and selection operator) / Squared Loss+L1

- The lasso is a shrinkage method like ridge, but acts in a nonlinear manner on the outcome y .
- The lasso is defined by

$$\sum_{i=1}^n (y_i - x_i^T \beta)^2$$

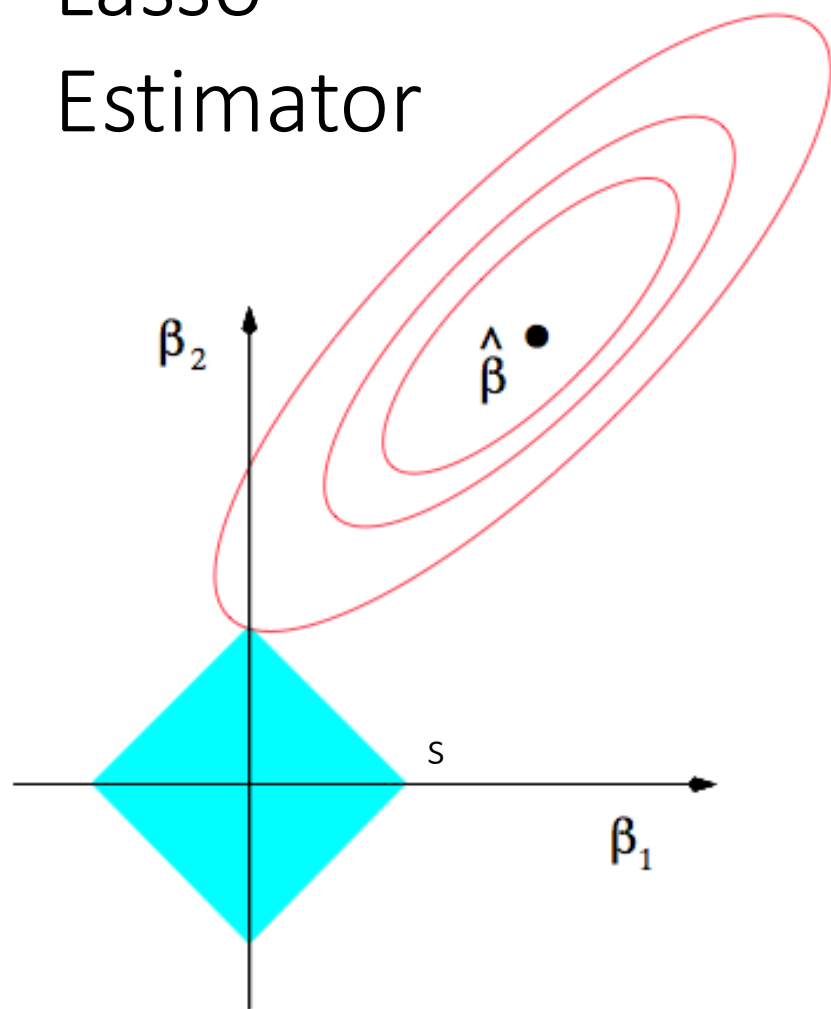

$$\hat{\beta}^{lasso} = \operatorname{argmin}_{\beta} (y - X\beta)^T (y - X\beta)$$

subject to $\sum_j |\beta_j| \leq s$

 L1 norm

By convention, the bias/intercept term is typically not regularized.
Here we assume data has been centered ... therefore no bias term

Lasso Estimator



Ridge Regression

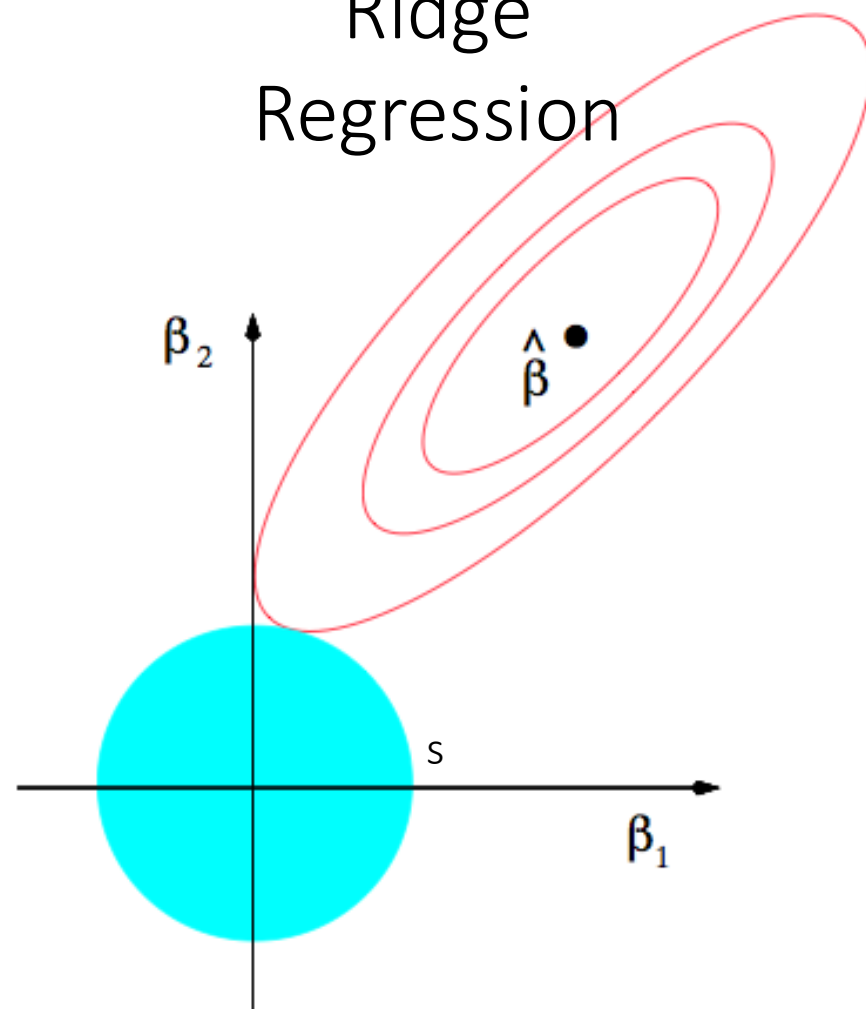
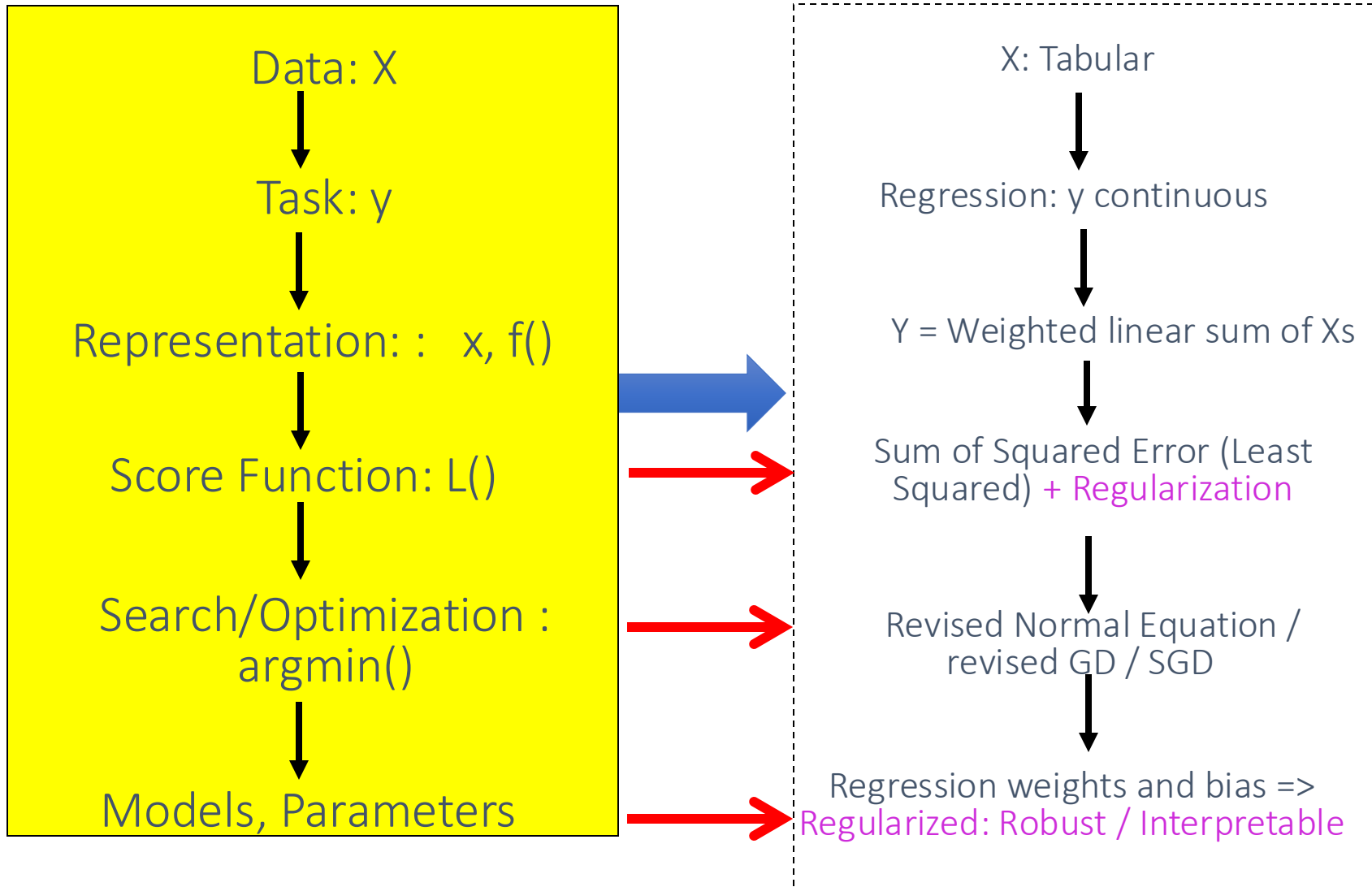


FIGURE 3.11. Estimation picture for the lasso (left) and ridge regression (right). Shown are contours of the error and constraint functions. The solid blue areas are the constraint regions $|\beta_1| + |\beta_2| \leq t$ and $\beta_1^2 + \beta_2^2 \leq t^2$, respectively, while the red ellipses are the contours of the least squares error function.

$$\min J(\beta) = \sum_{i=1}^n \left(Y - \hat{Y} \right)^2 + \lambda \left(\sum_{j=1}^p \beta_j^q \right)^{1/q}$$

Today: Regularized multivariate linear regression



More: A family of shrinkage estimators

$$\beta = \arg \min_{\beta} \sum_{i=1}^N (y_i - x_i^T \beta)^2$$

$$\text{subject to } \sum_j |\beta_j|^q \leq s$$

- for $q \geq 0$, contours of constant value of $\sum_j |\beta_j|^q$ are shown for the case of two inputs.

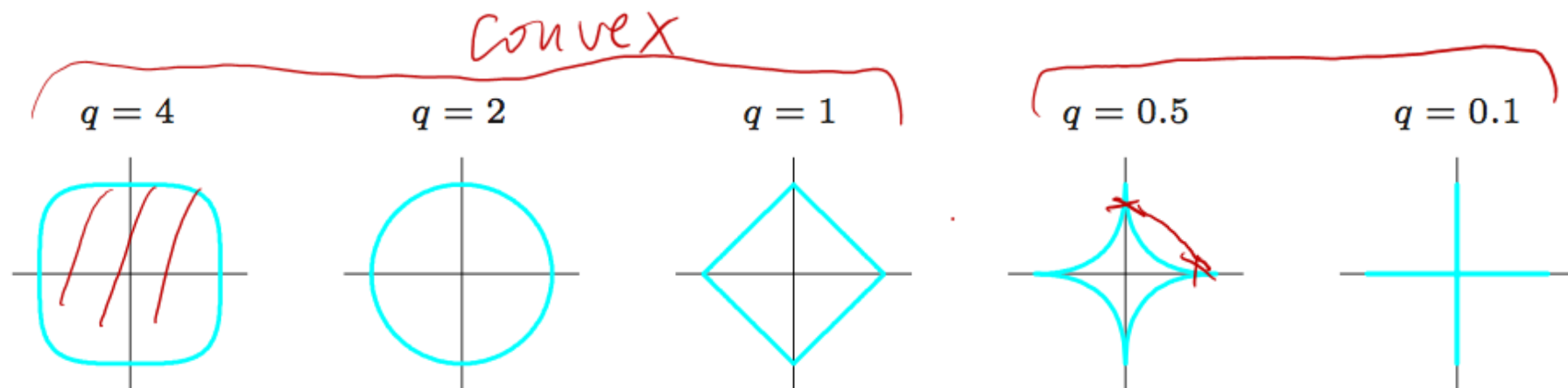
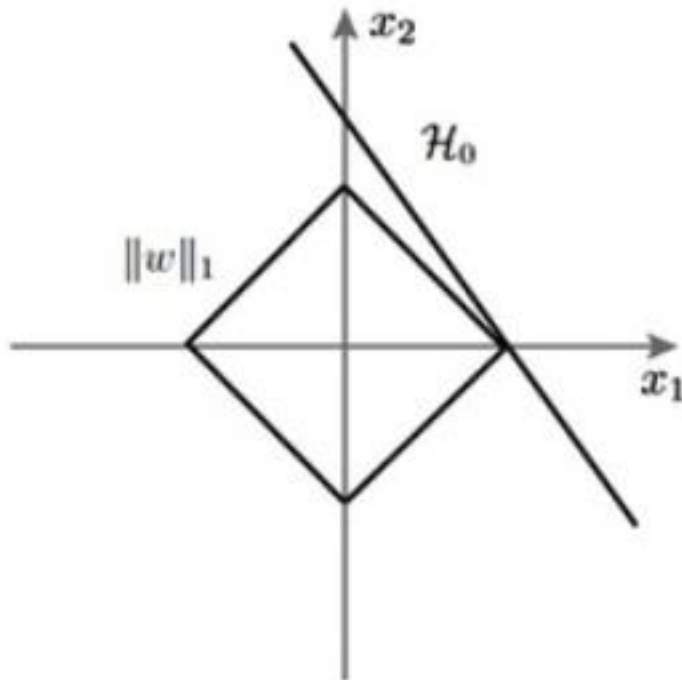
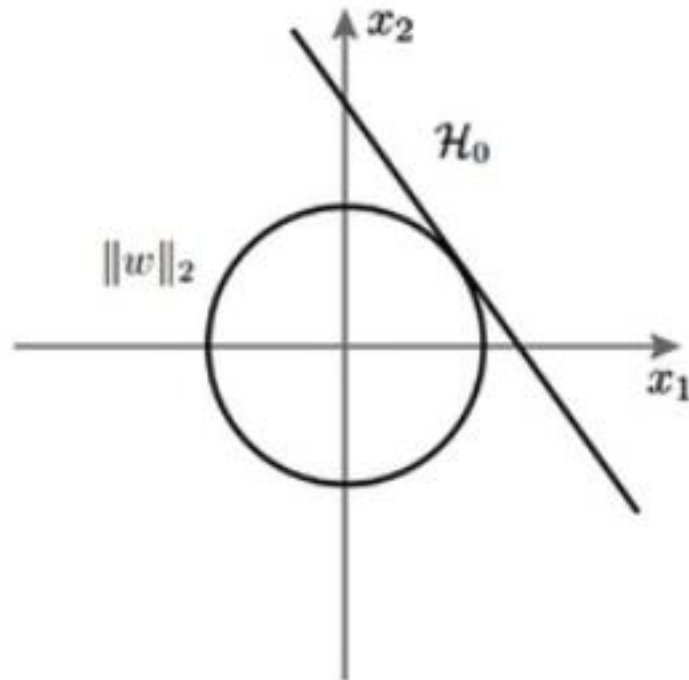


FIGURE 3.12. Contours of constant value of $\sum_j |\beta_j|^q$ for given values of q .

A L1 regularization



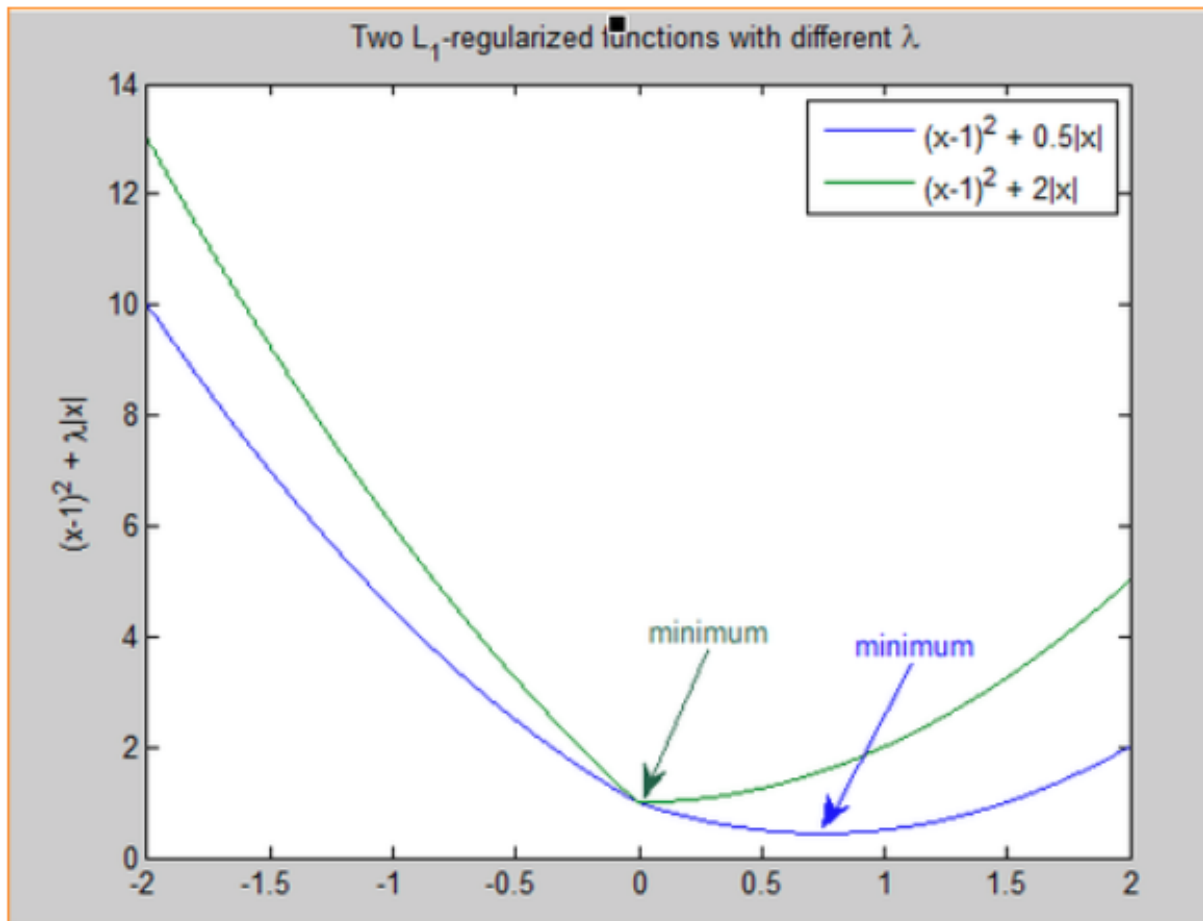
B L2 regularization



due to the nature of L₁ norm, the viable solutions are limited to corners, **which are on a few axis only**
- **in the above case x1. Value of x2 = 0.** This means that the solution has eliminated the role of x2, leading to sparsity

L_1 -regularized loss function $F(x) = f(x) + \lambda\|x\|_1$ is non-smooth. It's not differentiable at 0. Optimization theory says that the optimum of a function is either the point with 0-derivative or one of the irregularities (corners, kinks, etc.). So, it's possible that the optimal point of F is 0 even if 0 isn't the stationary point of f . In fact, it would be 0 if λ is large enough (stronger regularization effect). Below is a graphical illustration.

<http://www.quora.com/What-is-the-difference-between-L1-and-L2-regularization>



In mathematics, particularly in calculus, a stationary point or critical point of a differentiable function of one variable is a point of the domain of the function where the derivative is zero (equivalently, the slope of the graph at that point is zero).

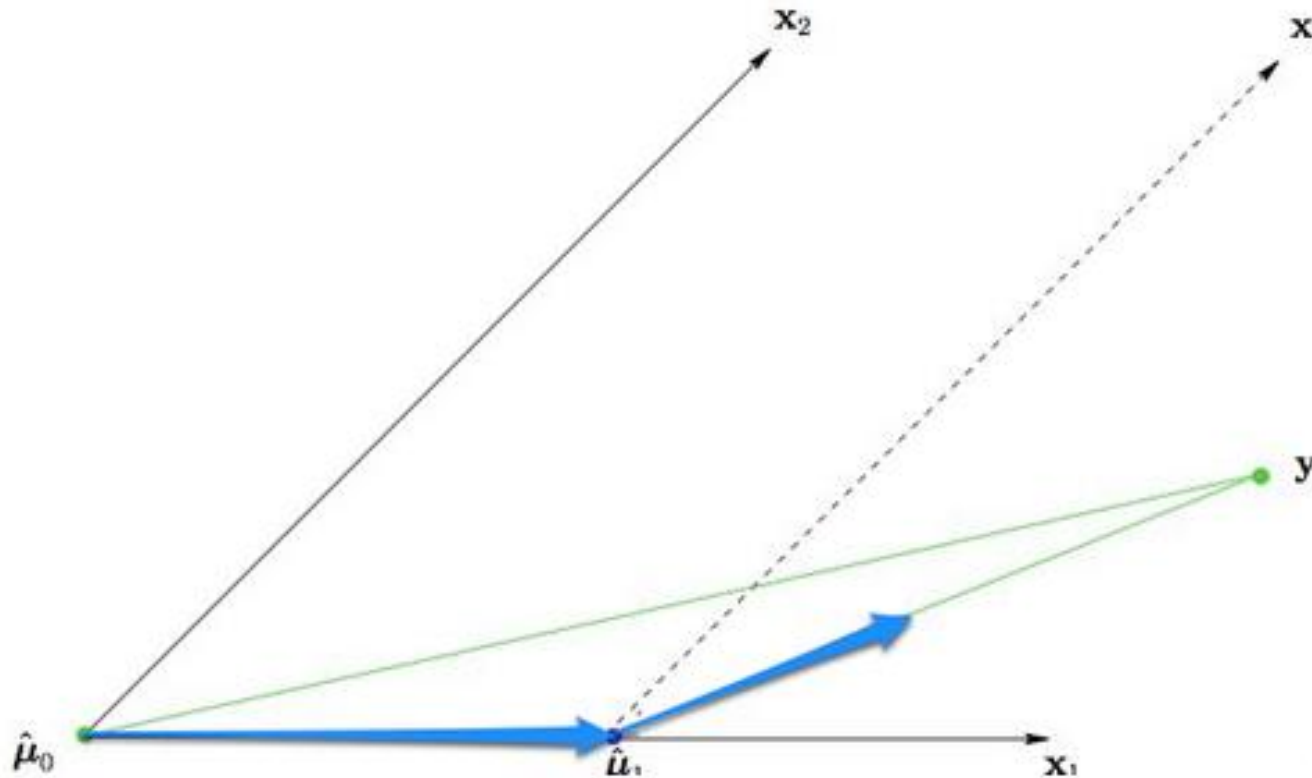
Coordinate descent based Learning of Lasso

1. Initialize β
2. Repeat until converged
3. For $j = 1, 2, \dots, P$ do
 - $a_j = 2 \sum_{i=1}^n x_{ij}^2$
 - $e_j = 2 \sum_{i=1}^n x_{ij} (y_i - x_i^T \beta + x_{ij} \beta_j)$
 - if $e_j < -\lambda$
$$\beta_j = (e_j + \lambda) / a_j$$
 - else if, $e_j > \lambda$
$$\beta_j = (e_j - \lambda) / a_j$$
 - else, soft-thresholding
$$\beta_j = 0$$

Coordinate descent (WIKI) → one does line search along one coordinate direction at the current point in each iteration.

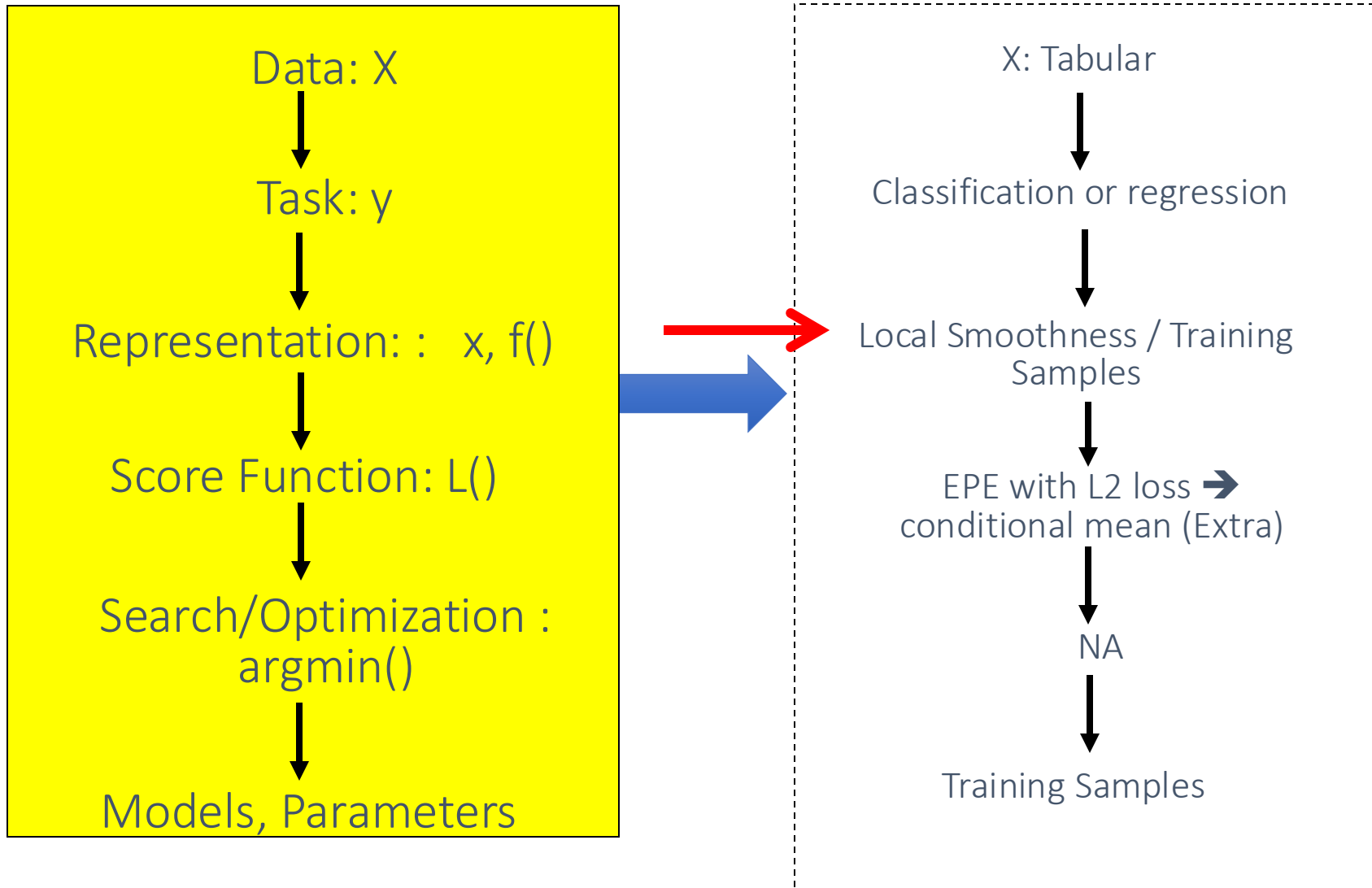
One uses different coordinate directions cyclically throughout the procedure.

Least Angle Regression (LARS) (State-of-the-art LASSO solver)



<http://statweb.stanford.edu/~tibs/ftp/lars.pdf>

L8: K-nearest-neighbor(regressor or classifier)



Code run: https://github.com/qiyanjun/2025Fall-UVA-CS-MachineLearningDeep/blob/main/notebook/L8_Knearest.ipynb

```
[42] # import regressor
      from sklearn.neighbors import KNeighborsRegressor
      # instantiate with K=5
      knn = KNeighborsRegressor(n_neighbors=5)
      # fit with data
      knn.fit(X, y)
```

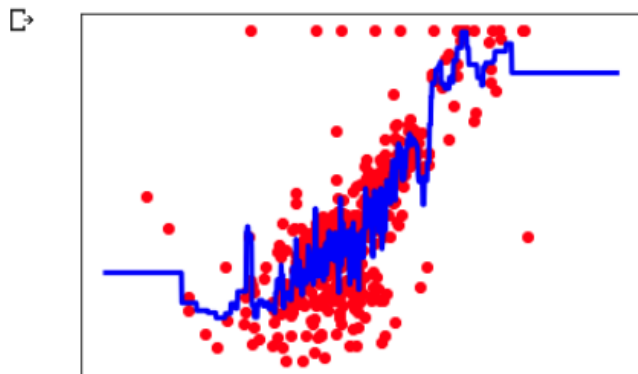
```
↳ KNeighborsRegressor(algorithm='auto', leaf_size=3
                       metric_params=None, n_jobs=No
                       weights='uniform')
```

```
###
Xfit = np.linspace(3, 10, 1000).reshape(-1, 1)
yfit = knn.predict(Xfit)

# Plot outputs
plt.scatter(X, y, color='red')
plt.plot(Xfit, yfit, color='blue', linewidth=3)

plt.xticks(())
plt.yticks(())

plt.show()
```



```
[44] # import regressor
      from sklearn.neighbors import KNeighborsRegressor
      # instantiate with K=5
      knn = KNeighborsRegressor(n_neighbors=100)
      # fit with data
      knn.fit(X, y)
```

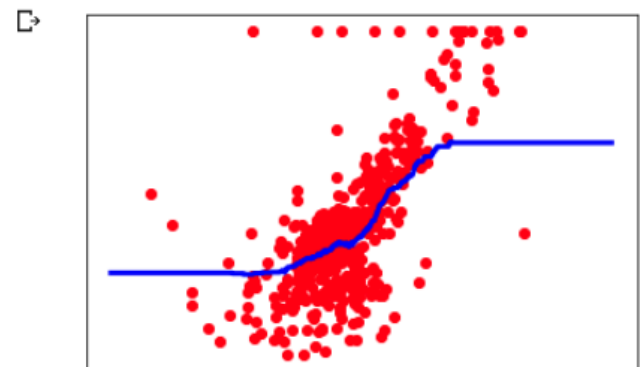
```
↳ KNeighborsRegressor(algorithm='auto', leaf_size=3(
                       metric_params=None, n_jobs=Nor
                       weights='uniform')
```

```
###
Xfit = np.linspace(3, 10, 1000).reshape(-1, 1)
yfit = knn.predict(Xfit)

# Plot outputs
plt.scatter(X, y, color='red')
plt.plot(Xfit, yfit, color='blue', linewidth=3)

plt.xticks(())
plt.yticks(())

plt.show()
```



K Nearest neighbor (Testing Mode)

It Needs:

1. The set of stored training samples
2. Distance metric to compute distance between samples
3. The value of k , i.e., the number of nearest neighbors to retrieve

Training Mode:

- (Naïve) version: **DO NOTHING !!!!**

Testing Model: To classify **unknown** sample:

- **Step1**: Compute distance to **all** training records
- **Step2**: Identify **k nearest** neighbors
- **Step3**: Use class labels of nearest neighbors to determine the class label of unknown record (**e.g., by taking majority vote**)

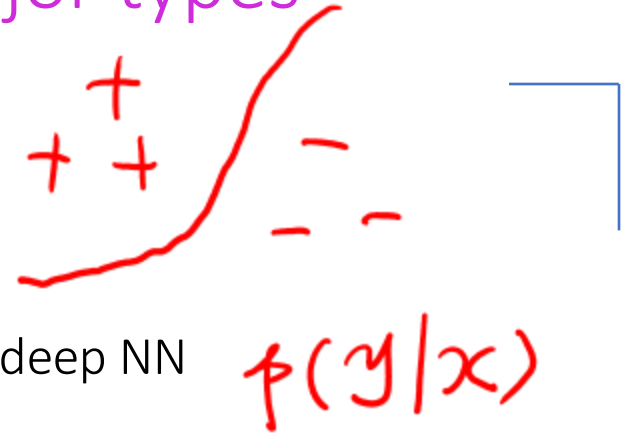
We can divide the large variety of supervised classifiers into roughly three major types

1. Discriminative

directly estimate a decision rule/boundary

e.g., support vector machine, decision tree,

e.g. [logistic regression](#), neural networks (NN), deep NN



2. Generative:

build a generative statistical model

e.g., Bayesian networks, [Naïve Bayes classifier](#)

$$P(x|y=c)$$



3. Instance based classifiers

- Use observation directly (no models)
- e.g. K nearest neighbors

Less
complex

More
complex

① Polynomial Regression

d

$d \downarrow$

$d \uparrow$

② Ridge Reg

λ
 $\lambda > 0$

$\lambda \uparrow$

$\lambda \downarrow$

③ KNN Reg

k

$k \uparrow$

$k \downarrow$

$k \downarrow$

Model Selection for Nearest neighbor classification

- Choosing the value of k :
 - If k is too small, sensitive to noise points
 - If k is too large, neighborhood may include points from other classes

- Bias and variance tradeoff

- A small neighborhood $\rightarrow$ large variance $\rightarrow$ [unreliable] estimation

- A large neighborhood $\rightarrow$ large bias $\rightarrow$ [inaccurate] estimation

overfit

underfit

We aim to make our trained model

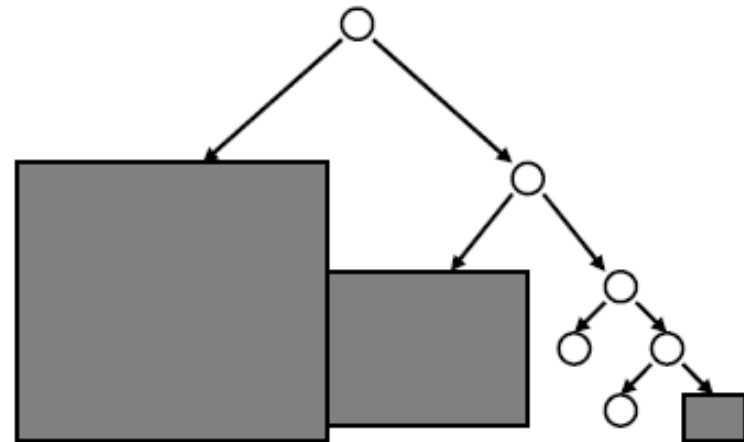
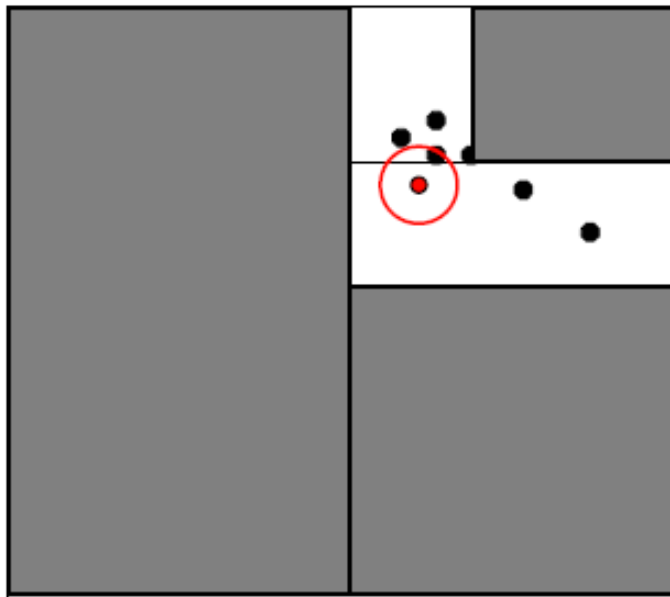
- 1. Generalize Well
- 2. Computational Scalable and Efficient
- 3. Trustworthy: Robust / Interpretable
 - Especially for some domains, this is about trust!

Computational Time Cost

	Train (n)	Test ($m=1$) $\hat{y} = \beta^T x$
Linear Regression	$O(np^2 + p^3)$	$O(p)$
KNN Reg	$O(1)$	$O(np) +$ $O(\text{sort } n-k)$???

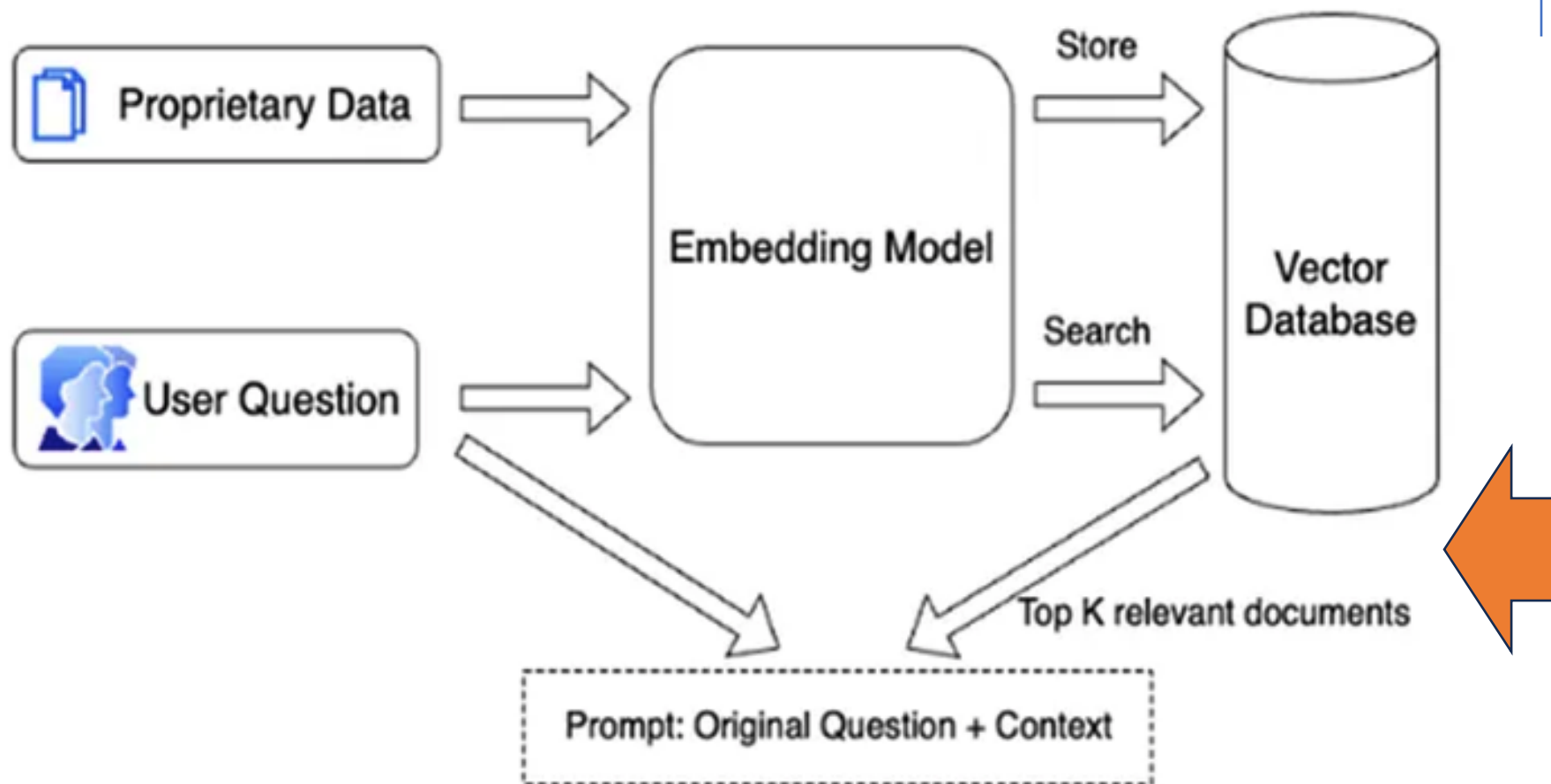
$p = 30,000$
 $n = 20,000$

NN Search by KD Tree



Using the distance bounds and the bounds of the data below each node, we can prune parts of the tree that could NOT include the nearest neighbor.

KNN as the Most critical component in Retrieval Augmented Generation System, e.g.:



09/30/2025 Roadmap

- TA to go over HW1
- One UVA ML club to introduce their setups and projects
- Q5
- Review Q4
- Review QA for L5-L7

Q4 review

1. True or False? In a linear regression model, the number of parameters increase as the number of samples increase



Multiple choice

☐ True

☐ False

☐ Add option or [add "Other"](#)



Answer key (2 points)



Required





2. Which data split should the model never be trained on? ()



Multiple choice

☐ Training Set

☐ K fold Split

☒ Test Set



☐ Add option or [add "Other"](#)



Answer key (2 points)



Required



3. Which of the following is an advantage of k-fold cross validation over the simple validation set method for model selection?



Multiple choice

- ☐ Requires fewer computations
- ☐ Provides a lower-variance estimate of model performance
- ☐ Uses less training data overall
- ☐ Always achieves lower test error
- ☐ Add option or [add "Other"](#)



Answer key (2 points)



Required





4. Suppose a model has low training error but very high test error. Which of the following strategies could help? (check all that apply)



Checkboxes

☐ Use a more complex learner

☐ Add more training data



☐ Reduce the number of features



☐ Add option or [add "Other"](#)



Answer key (2 points)



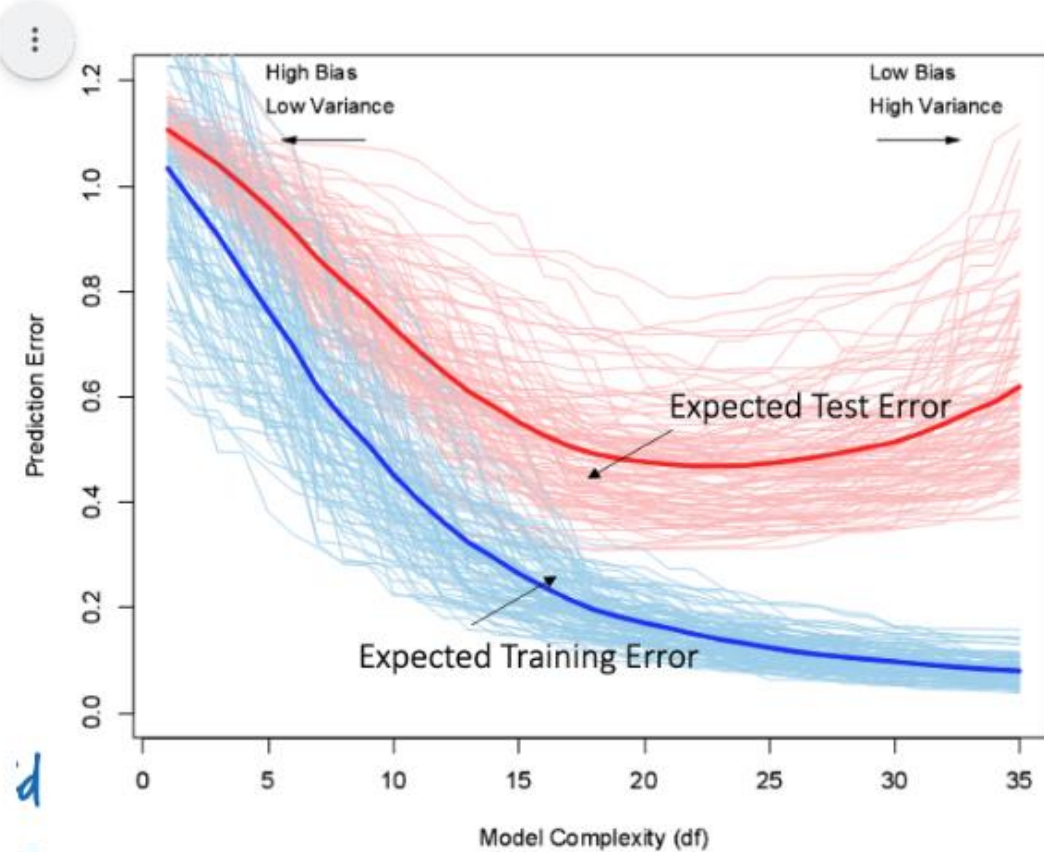
Required



5. Consider the below diagram. If you have a model with complexity of 5 degrees of freedom ($d=5$), it will face:



☒ Multiple choice



☐ Underfitting

☐ Overfitting

☐ Neither

☐ Add option or add "Other"



09/30/2025 Roadmap

- TA to go over HW1
- One UVA ML club to introduce their setups and projects
- Q5
- Review Q4
- Review QA for L5-L8

Questions on L5-L8

Set 1: Bias–Variance, Overfitting, and Model Complexity

- How do bias and variance contribute to generalization error, and how do we find the “sweet spot” without knowing the true distribution?
- What are practical indicators of underfitting vs. overfitting (from graphs, learning curves, or error plots), and how do we fix each?
- How does cross-validation (choice of K) approximate generalization error, and what are the trade-offs (bias vs variance, LOOCV vs k -fold)?
- Why does zero training error often generalize poorly, and how is this linked to variance?
- Would we ever prefer high bias or high variance, and how do we reduce one while controlling the other?

Set 2: Regularization (LASSO, Ridge, Elastic Net, Generalizations)

- What are the key differences between L1 (LASSO), L2 (Ridge), and Elastic Net in terms of sparsity, robustness, computational cost, and when to use each?
- Why do L1 penalties set coefficients to zero, while L2 does not? What happens when $p > n$ or when features are highly correlated?
- How does the choice of λ affect bias–variance, and how do we select it (cross-validation, validation curves)?
- Are there equivalent closed-form solutions for LASSO like Ridge has? Why are L1 and L2 chosen—what about higher-order penalties?
- When is Elastic Net preferable (e.g., grouped correlated features), and can we always default to it?

Questions on L5-L8

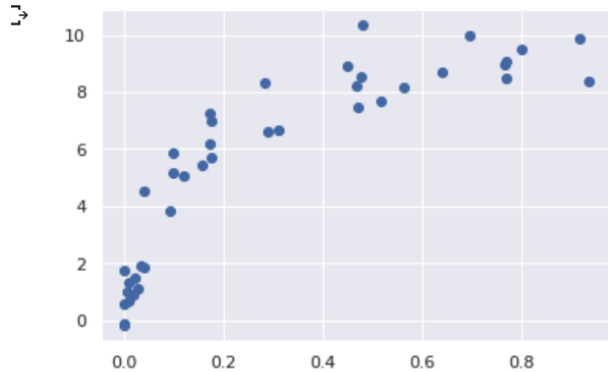
Set 3: k-Nearest Neighbors (kNN) and Instance-Based Learning

- How do we pick the best k (odd vs even, weighted vs unweighted, trade-offs with noisy data)?
- What is the computational cost of kNN (sorting term, memory cost), and can it overfit?
- How does the distance metric affect performance, and how are ties handled in classification?
- What are the advantages/disadvantages of kNN vs gradient descent or regularized linear models?
- In practice, how large must the dataset be to offset outliers, and is kNN more effective for regression or classification?

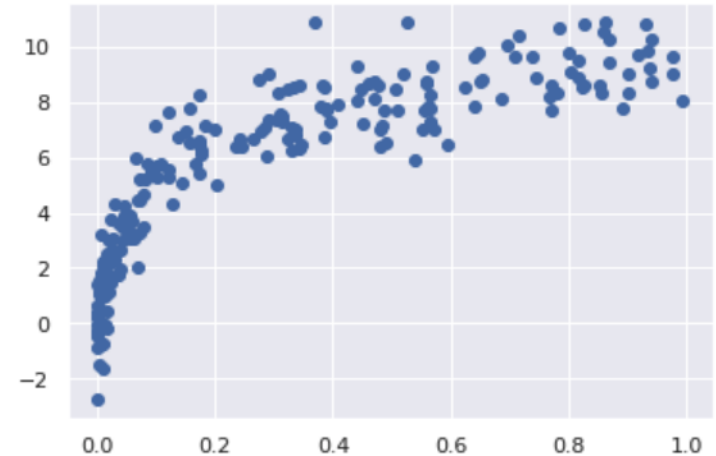
Set 4: Maximum Likelihood Estimation (MLE) and Probability Foundations

- Why do we usually maximize the log-likelihood instead of the likelihood itself, and how does this connect to squared error in linear regression?
- How does MLE extend from discrete distributions (e.g., coin flips) to continuous (e.g., Gaussians)?
- Why is the MLE for Bernoulli just the sample proportion, and what happens with small samples or noisy data?
- What makes MLE consistent and efficient, and are there situations where maximum likelihood may not yield the most “ideal” parameter?
- How does the bias–variance decomposition change with different loss functions (e.g., 0–1 loss, Laplace errors)?

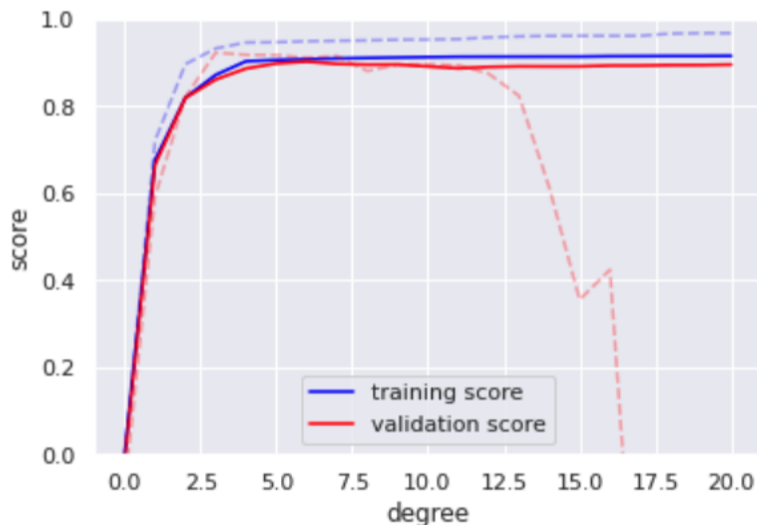
```
X, y = make_data(40)
plt.scatter(X, y);
```



```
X2, y2 = make_data(200)
plt.scatter(X2.ravel(), y2);
```



```
plt.plot(degree, np.median(train_score2, 1), color='blue',
plt.plot(degree, np.median(val_score2, 1), color='red', 1
plt.plot(degree, np.median(train_score, 1), color='blue',
plt.plot(degree, np.median(val_score, 1), color='red', al
plt.legend(loc='lower center')
plt.ylim(0, 1)
plt.xlabel('degree')
plt.ylabel('score');
```



Behavior of the validation curve:

- the model complexity
- the number of training points

```
X2, y2 = make_data(200)
```

```
degree = np.arange(200)
```

```
train_score2, val_score2 = validation_curve(PolynomialRegression, X2, y2, degree, 'polynomial')
```

```
plt.plot(degree, np.median(train_score2, 1), color='blue', label='training score')
plt.plot(degree, np.median(val_score2, 1), color='red', label='validation score')
plt.legend(loc='lower center')
plt.ylim(0, 1)
plt.xlabel('degree')
plt.ylabel('score');
```

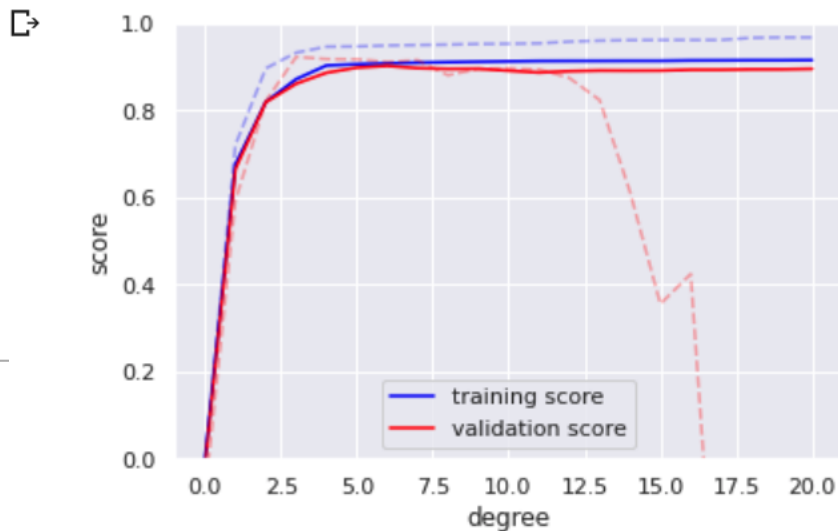


```
X2, y2 = make_data(200)
```

```
degree = np.arange(21)
```

```
train_score2, val_score2 = validation_curve(PolynomialRegression, X2, y2, degree, 'polynomial')
```

```
plt.plot(degree, np.median(train_score2, 1), color='blue', label='training score')
plt.plot(degree, np.median(val_score2, 1), color='red', label='validation score')
plt.plot(degree, np.median(train_score, 1), color='blue', label='training score')
plt.plot(degree, np.median(val_score, 1), color='red', label='validation score')
plt.legend(loc='lower center')
plt.ylim(0, 1)
plt.xlabel('degree')
plt.ylabel('score');
```



Interesting Relation between

- the right range of model complexity
- the number of training points